

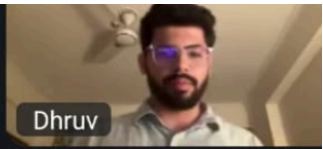
Searching

Searching

- to look for something
- to find something

$A = [1, 10, 7, 6, 4, 3, 2, 11, 9, 15, 17, 28]$

Double-click or press **Enter** to edit text



Dhruv

1. Linear Search

Linear Search

```
A = [10, 7, 6, 4, 3, 2, 11, 9, 15, 17, 28]
target = 9

bool flag = false;

for(i = 0; i < N; i++)
{
    if(A[i] == target)
    {
        flag = true;
        break;
    }
}

if(flag)
{
    cout << "YES";
}
else
{
    cout << "NO";
}
```

More optimized version of above.

$A = [2, 3, 4, 6, 7, 9, 10, 11, 15, 17, 28]$

target = 29

Data is sorted, we can break early as well

```
bool flag = false;

for(i = 0; i < N; i++)
{
    if(A[i] == target)
    {
        flag = true;
        break;
    }

    if(A[i] > target)
    {
        break;
    }
}

if(flag)
{
    cout << "YES";
}
else
{
    cout << "NO";
}
```

2. Binary Search

SORTED DATA => BINARY SEARCH

l
 r
 $|$

$A = [2, 3, 4, 6, 7, 9, 10, 11, 15, 17, 28, 35, 40, 60, 65, 69, 70]$
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

target = 60

mid

•

l	r	$mid = (l + r) / 2$
0	16	$(0 + 16) / 2 = 8$
9	16	$(9 + 16) / 2 = 25 / 2 = 12$
13	16	$(13 + 16) / 2 = 29 / 2 = 14$
13	13	$(13 + 13) / 2 = 13$

if $A[mid] == \text{target}$:
 flag = true;
 break;

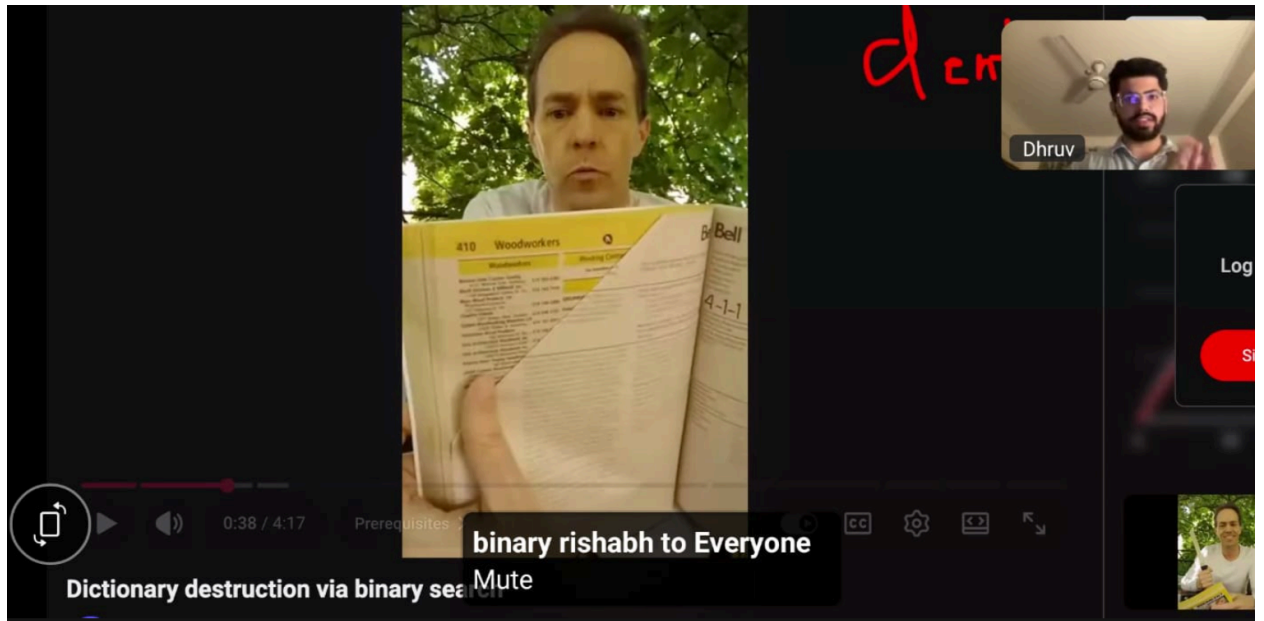
else if $A[mid] > \text{target}$:
 // search in the Left

else:
 // search in the Right

$A = [2, 3, 4, 6, 7, 9, 10, 11, 15, 17, 28, 35, 40, 60, 65, 69, 70]$
0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16

target = 52

l	r	$mid = (l + r) / 2$	
0	16	$(0 + 16) / 2 = 8$	$l = mid + 1$
9	16	$(9 + 16) / 2 = 25 / 2 = 12$	$l = mid + 1$
13	16	$(13 + 16) / 2 = 29 / 2 = 14$	$r = mid - 1$
13	13	$(13 + 13) / 2 = 26 / 2 = 13$	



Binary Search code

```
flag = false
```

```
l = 0
```

```
r = N - 1
```

```
while l <= r:
```

```
    mid = (l + r) / 2
```

```
    if A[mid] == target:
```

```
        flag = true;
```

```
        break;
```

```
    else if A[mid] > target:
```

```
        // search in the left
```

```
        r = mid - 1;
```

```
    else:
```

```
        // search in the right
```

```
        l = mid + 1
```

```
if(flag)
```

```
{
```

```
    cout << "YES";
```

```
}
```

```
else
```

```
{
```

```
    cout << "NO";
```

```
}
```

Diagram illustrating the array A and indices l , r , and m .

Array A is shown with elements $1, 7, 9, 11, 15$ at indices $0, 1, 2, 3, 4$ respectively. The element 9 at index 2 is highlighted with a red underline. Above the array, the indices l , r , and m are indicated, with r pointing to index 2 and m pointing to index 3 .

target = 9

Sorting

SORTING

- To rearrange data in a specific order
- Ascending / Descending

$A = [1, 2, 4, 8, 10, 20, 30, 40]$ Sorted in asc order

$A = [100, 50, 30, 10]$ Sorted in desc order

In this lecture he will cover only selection sort, bubble sort, insertion sort and merge sort, heap sort, quick sort he will do in future

$A = [11, 2, 14, 18, 20, 27, 3, 40]$

Goal -> sort in asc order

[1) Selection sort
2) Bubble sort
3) Insertion sort **]**

4) Merge sort
5) Heap sort
6) Quick sort

1. Selection sort

- 1) Select the min
in the unsorted region
- 2) Bring to the front
by swapping it with
the first element
of unsorted region

$A = [11, 2, 14, 18, 20, 27, 3, 40]$

$A = [2, 11, 14, 18, 20, 27, 3, 40]$

$A = [2, 3, 14, 18, 20, 27, 11, 40]$

$A = [2, 3, 11, 18, 20, 27, 14, 40]$

$A = [2, 3, 11, 14, 20, 27, 18, 40]$

$A = [2, 3, 11, 14, 18, 27, 20, 40]$

$A = [2, 3, 11, 14, 18, 20, 27, 40]$

$A = [2, 3, 11, 14, 18, 20, 27, 40]$

Code of selection sort

```
for(i = 0; i < N; i++)
{
    // find min in the unsorted region
    // [i, N - 1] / [i, N)

    int mn = A[i];
    int loc = i;
    for(int j = i + 1; j < N; j++)
    {
        if(A[j] < mn)
        {
            mn = A[j];
            loc = j;
        }
    }

    swap(A[i], A[loc]);
}
```


Bubble sort

Bubble Sort

- If two neighbours / adjacent element are in the wrong order => swap them

$A = [2, 11, 14, 18, 20, 3, 27, 40]$

$A = [2, 11, 14, 18, 3, 20, 27, 40]$

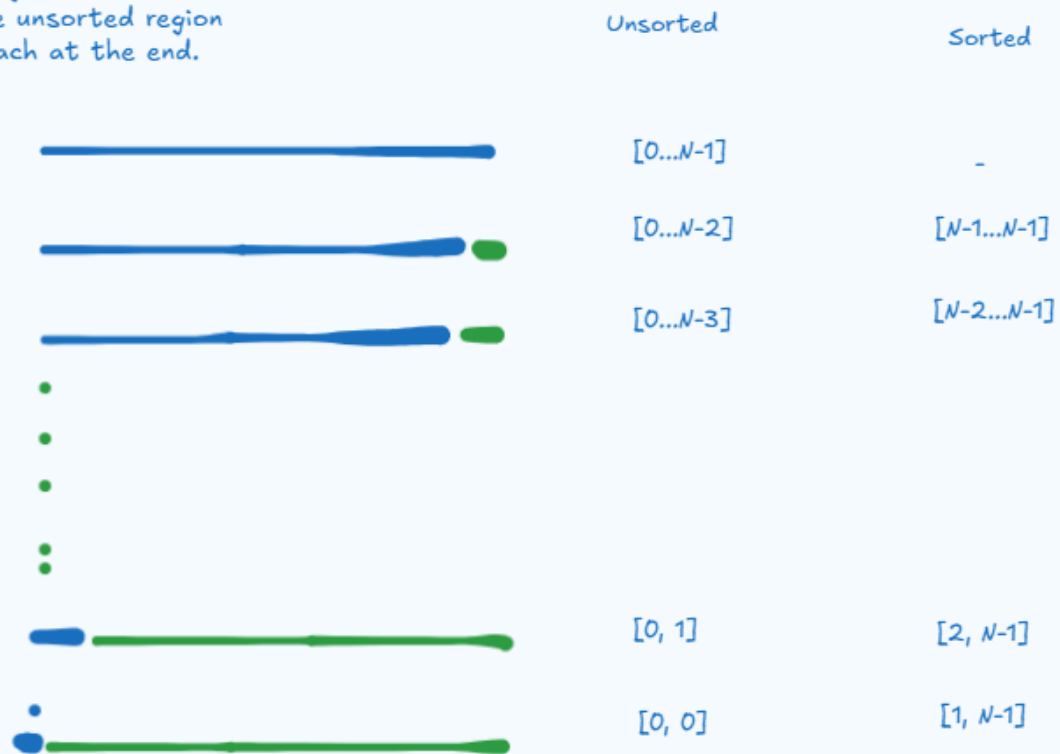
$A = [2, 11, 14, 3, 18, 20, 27, 40]$

$A = [2, 11, 3, 14, 18, 20, 27, 40]$

$A = [2, 3, 11, 14, 18, 20, 27, 40]$

$A = [2, 3, 11, 14, 18, 20, 27, 40]$

After every pass,
the largest element
of the unsorted region
will reach at the end.



Code of bubble sort

```
for(i = N - 1; i >= 1; i--)
{
    // [0, i] is unsorted
    // swap there

    for(j = 0; j < i; j++)
    {
        if(A[j] > A[j + 1])
        {
            swap(A[j], A[j + 1]);
        }
    }
    // [0, i] -> is unsorted
}
```

We can break early, if no swap happened

=> array was already sorted

Below is code(optimized version of bubble sort)

```
for(i = N - 1; i >= 1; i--)  
{  
  
    // [0, i] is unsorted  
    // swap there  
  
    bool swapped = false;  
  
    for(j = 0; j < i; j++)  
    {  
        if(A[j] > A[j + 1])  
        {  
            swapped = true;  
            swap(A[j], A[j + 1]);  
        }  
    }  
  
    if(swapped == false)  
    {  
        break;  
    }  
}
```